

Calgary SAS User Group Meeting

Optimization of Resources with SAS:

A Case of Oil Processing Business Maximizing Profit From Competing Opportunities

Eugene.Yankovsky@sas.com

Technical Business Analyst,

Risk Advisory, a Division of SAS Institute (Calgary)

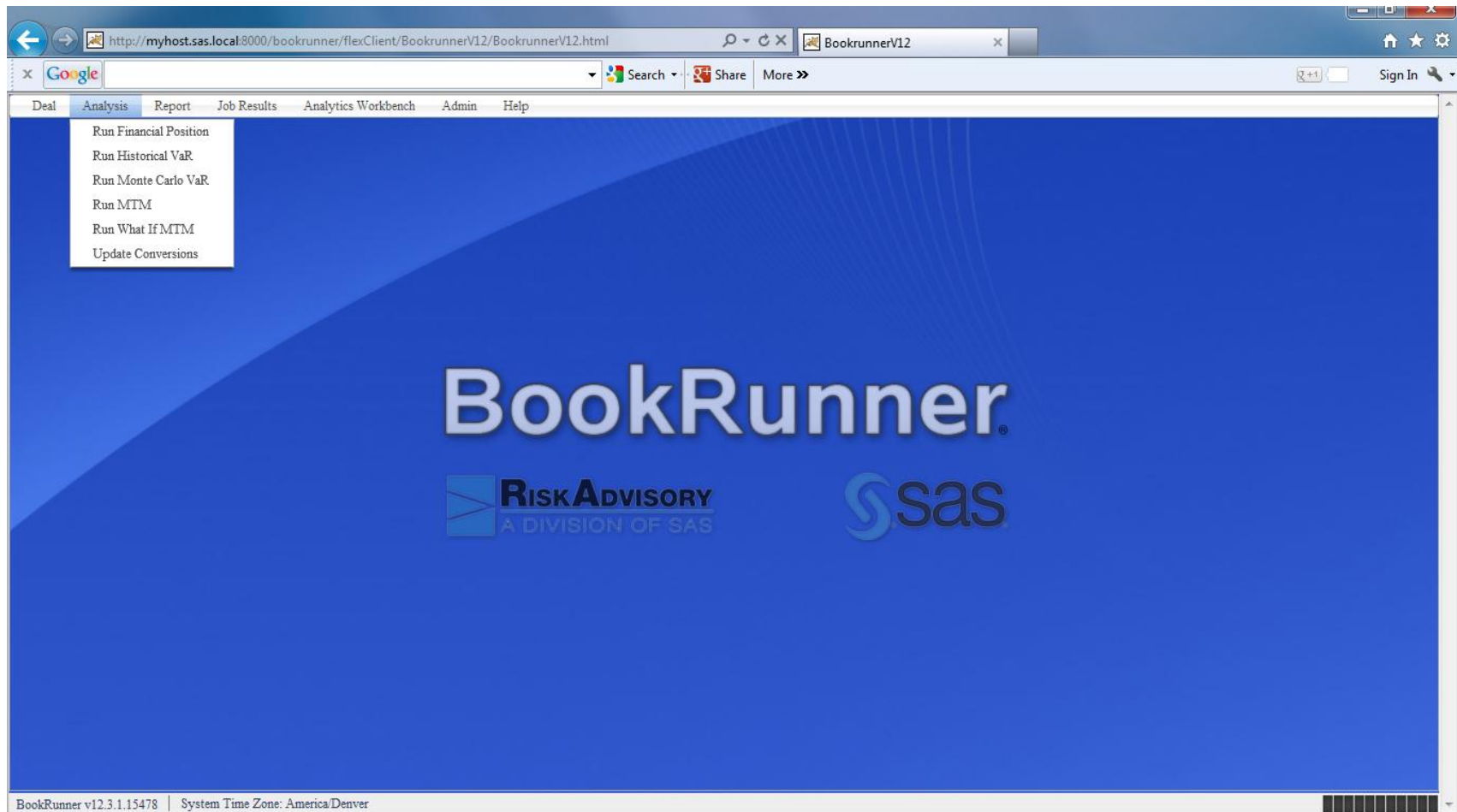


November 8th, 2013

Agenda:

1. Introduction
2. Motivation
3. Challenges
4. Solutions
5. Future improvements

Introduction

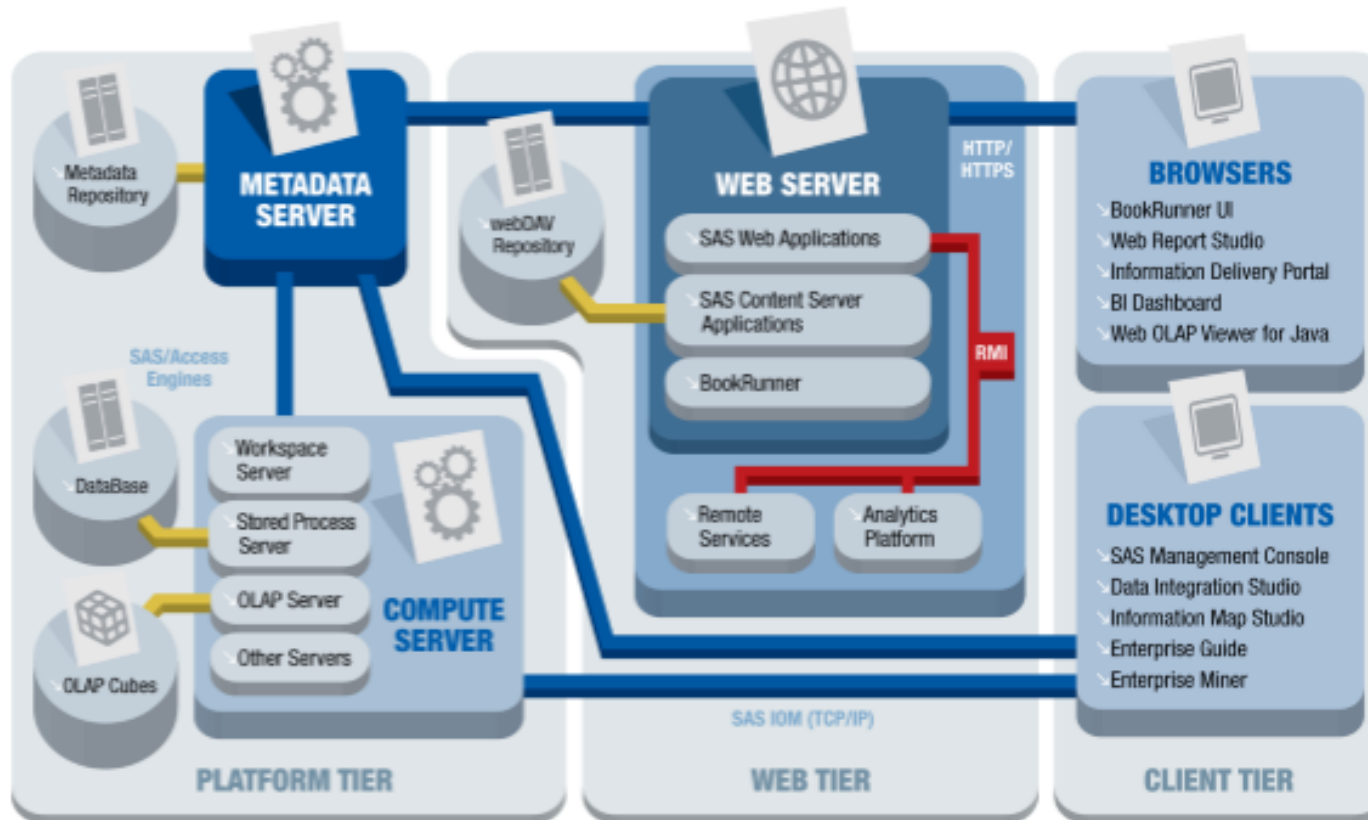


More information can be found at:

<http://www.riskadvisory.com/bookrunneradvancedanalytics.htm>

<http://www.sas.com/solutions/riskmgmt/bookrunner/>

BR v12 Architecture Leverages SAS 9.2 Platform



Motivation:

- 1. Share solution that can be applied to any manufacture or service with the processing data**

Scheme. Manufacture or Service



Dual optimization problems:

Maximize profit (or the social value) subject to the input and processing constraints

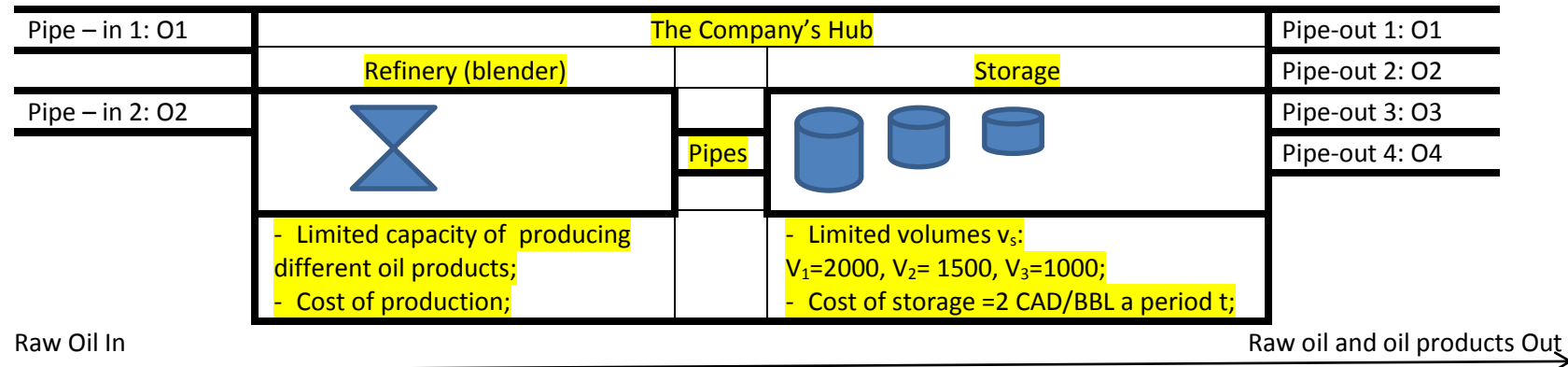
Minimize the expenses subject to fixed profit (social value) and processing constraints

- 2. Get feedback on the model for future improvements**

Challenge 1: Define the inputs, process and outputs' constraints

Solution 1: Use a simplifying model holding the key elements of the process:

Scheme. One time period snapshot



Oil processing business looking for a maximum profit by synchronizing

1. Reselling at spot price: cashing location spread
2. Blending and selling at spot price: cashing production margin
3. Storage and selling at forward price: cashing location and time spread
4. Blending, storage and selling at forward price: cashing production margin, location and time spread

Maximize total profit π_{Tot} on all inputs before optimization started for the next time period t :

$$\text{Maximize } \pi_{Tot} = \pi_{Blending} + \pi_{Storage}$$

$$\begin{cases} \pi_{Blending} = P_{out} * Y - P_{in} * (X - S + S_0) \\ \pi_{Storage} = S * ((P_{out}(t) - C_s * t) * e^{-r*t} - P_{in}) \end{cases}$$

With respect to the system arguments (objects for maximization):

X is a vector of inputs;

Y is a vector of final outputs;

S_0 is a vector of inventories of inputs and outputs before optimization started, $t = 0$.

S is a vector of inventories at the end of the of the optimization period, t ;

P_{in} is a vector of all **spot prices** on all inputs before optimization started, $t = 0$

P_{out} is a vector of spot prices the oil producer can sell outputs at the Hub at the end of the horizon, $t = 1$;

$P_{out}(t)$ is a $1 \times K$ vector of all outputs' forward prices the oil producer can sell outputs at the Hub at the futures time period t observed at the beginning of optimization;

t = time between now (assumed to be a zero time point) and time in the future t ;

C_s = cost of storing per one unit of measurement (BBL) per time instant;

r = "risk free" interest rate that is available to the oil producer.

Table 1. Naming conventions

Oil Product Name	Gross product notation	Final product out	Inventories at the beginning, before optimization	Inventories in the storage to hold for optimization period, t
Vector notation	X	Y	So	S
O1	X1	Y1	So1	S1
O2	X2	Y2	So2	S2
OO1	X3	Y3	So3	S3
OO2	X4	Y4	So4	S4

Numeric example input parameters

Oil Product	Input price CAD, BBL	Output Price	Forward Output price
	P_{in}	P_{out}	P_{out}(t)
O1	80	82	85
O2	82	84	87
OO1	N/a	300	320
OO2	N/a	190	200

Note:

N/a = the oil product is not produced upstream

We assume production cost as an input price for OO1 and OO2:

$$P_{3in} = P_1 \cdot (X_3/X_1) + P_2 \cdot (X_3/X_2) = \mathbf{244}$$

$$P_{4in} = P_1 \cdot (X_1/X_4) + P_2 \cdot (X_2/X_4) + P_3 \cdot (X_3/X_4) = \mathbf{162}$$

Transportation capacity constraints

x.2) Input X constrained by pipeline capacity Cap_in for the oil products to be shipped in:

$$X_{in} \leq Cap_in$$

Pipeline into a hub	Oil product shipped	Capacity Cap_in , BBL/month
1	O1	5000
2	O2	3000

y.1) Output constrained by pipeline non-exclusive capacity Cap_out for the oil products to be shipped out:

$$Y - S + S_0 \leq Cap_out$$

Pipeline out of a hub	Oil product shipped	Capacity Cap_out , BBL/month
1	Either product: O1, O2, O3, O4	4000
2		2000
3		1000
4		1000

Continuous annualized risk free rate of return (per month)

$r =$	0.005
-------	-------

Time period, $t = 1$ month

Challenge 2: Production relationship constraints

Solution 2.1 (theoretical): estimate the relationship assuming some classic form:

E.g., a modification of Cobb-Douglas model

$$E(Y_t) = \beta_0 * X_{1,t}^{\beta_1} * X_{2,t}^{\beta_2}$$

Or

$$E(\log Y_t) = \log \beta'_0 + \beta'_1 * \log X_{1,t} + \beta'_2 * \log X_{2,t}$$

Limitations:

1. A sample size of >10 observations per each input predictor X (n=30 time obs in example above) ;
⇒ Assuming stable technological process for all X and Y, and equipment;
2. Log-linear relationship between inputs and outputs;
3. Limitations for storage accounting

Solution 2.2 (practical): Leontieff's Input-Output Model of National Accounts (Nobel prize, 1936)¹:

Economic Activities	1	2	...	...	Z	Exports	Final Demand	Total Outputs
1								
2								
...								
...								
Z								
Imports								

$$x = Ax + d$$

x is a vector of total output,

A is a transformation matrix of a_{ij} coefficients representing

i th sector, in order to produce 1 unit, must use a_{ij} units from sector j .

Ax is an intermediate output;

d is a vector of final demand, then our expression for the economy becomes

$$x_i = a_{1i}x_1 + a_{2i}x_2 + \dots + a_{ni}x_n + d_i,$$

¹ See more details on the model in http://en.wikipedia.org/wiki/Input%E2%80%93output_model

x.1) In this case production equation system using the P&L and Ledger accounts:

$$X = AX + d = AX + Y + (S - S_0)$$

X = a vector of total outputs (in volume units or in \$);

AX = inputs consumed by manufacture (in volume units or in \$);

A = matrix of the direct production expenses;

Y = a vector of final oil products to be shipped out (in volume units or in \$);

$(S - S_0)$ = change in inventories of oil stored in the storage tanks (in volume units or in \$).

Straight solution:

$$X = AX + (Y + (S - S_0))$$

$$(I - A) * X = (Y + (S - S_0))$$

$$X = (I - A)^{-1} * (Y + (S - S_0))$$

Requiring existence of the inverse matrix $(I - A)^{-1}$

SAS advantages:

- ⇒ Variety of built-in approximation methods if $(I - A)^{-1}$ does not exist
- ⇒ Can handle large number of inputs.

Table. Production/operational balance for the last period (month or quarter).

	Cost (resources consumed by production)				Total Output calculation			
Oil Products	O1	O2	OO1	OO2	Stored inventories(0)	Final product out	Stored inventories	Total Output
	X1	X2	X3	X4	S0	Y	S	$X=Y+(S-S_0)$
O1	0	0	200	300	100	500	100	500
O2	0	0	800	200	0	1000	0	1000
OO1	0	0	0	100	200	200	200	200
OO2	0	0	0	0	100	400	100	400
Total Output (X)	500	1000	200	400				

Transformation Matrix, A

Inputs	Outputs			
	O1 (X1)	O2 (X2)	OO1 (X3)	OO2 (X4)
O1 (X1)	0 (-> Y1)	0	1	0.75
O2 (X2)	0	0 (-> Y2)	4	0.5
OO1 (X3)	0	0	0	0.25
OO2 (X4)	0	0	0	0

System of production equations accounting for the initial oil inventories $S_0 = \{100, 0, 200, 100\}$:

$$\mathbf{X} = \mathbf{AX} + \mathbf{Y} + (\mathbf{S} - \mathbf{S}_0)$$

$$\text{O1: } X_1 = 0 \cdot X_1 + 0 \cdot X_2 + 1 \cdot X_3 + 0.75 \cdot X_4 + Y_1 + (S_1 - S_{10})$$

$$\text{O2: } X_2 = 0 \cdot X_1 + 1 \cdot X_2 + 2 \cdot X_3 + 0.5 \cdot X_4 + Y_2 - (S_2 - S_{20})$$

$$\text{OO1: } X_3 = 0 \cdot X_1 + 0 \cdot X_2 + 0 \cdot X_3 + 0.25 \cdot X_4 + Y_3 - (S_3 - S_{30})$$

$$\text{OO2: } X_4 = 0 \cdot X_1 + 0 \cdot X_2 + 0 \cdot X_3 + 0 \cdot X_4 + Y_4 - (S_4 - S_{40})$$

Challenge 3: Storage restrictions “Oil products do not mix”

Solution 3: Apply trial 1st, restrictive runs 2nd, and select solution with maximum profit.

Storage capacity limit is **three** tanks $\text{Cap}_s = \{2000, 1500, 1000\}$.

Work algorithm:

1.1. Trial run with flexible storage:

$$\mathbf{Max} \pi_{Tot} = \pi_B + \pi_S$$

Subject to

$$S1 + S2 + S3 + S4 \leq (\text{Cap}_{s1} + \text{Cap}_{s2} + \text{Cap}_{s3})$$

1.2. No-flexibility runs trying combinations of exclusive restrictions following the principles:

- a) test different storage constraints that are as close to the trial run's results;
- b) allocate more storage space to the product with higher gradient of the profit function;

$$\mathbf{Max} \pi_{Tot} = \pi_B + \pi_S$$

Subject to either exclusive storage constraints

See details in the example

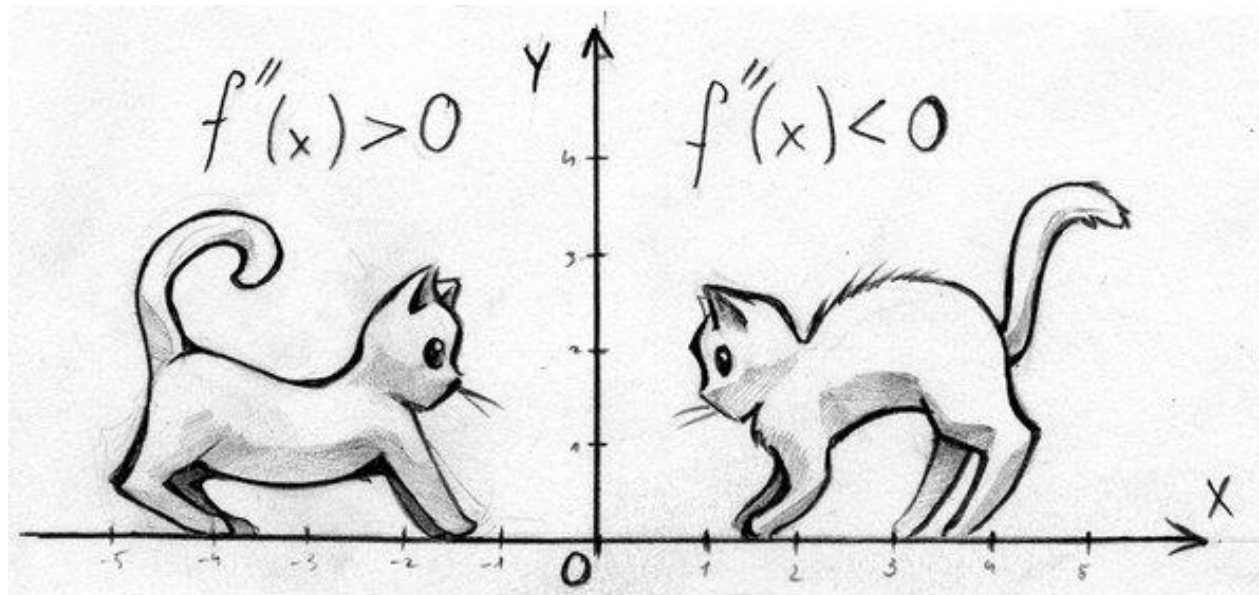
Solution in SAS:

PROC NLP offers functionality for finding local and global optimums.

E.g, minimum

$$\begin{array}{ll} \min_{x \in \mathcal{S}^n} & f(x) \\ \text{subject to} & c_i(x) = 0, \quad i = 1, \dots, m_e \\ & c_i(x) \geq 0, \quad i = m_e + 1, \dots, m \end{array}$$

where f is the objective function projected into in feasible region $\mathcal{G}$ and the c_i 's are the constraint functions



In local minimum x^* :

1. The gradient (vector of first derivatives) of the objective function f at the point x^* :

$$g(x^*) = \nabla f(x^*) = 0.$$

2. The Hessian (matrix of second derivatives) of the objective function at the point x^* is positive definite

$$G(x^*) = \nabla^2 f(x^*) > 0$$

In global optimum, x^* :

PROC NLP offers options for selecting different starting points either at random or by selecting a point on a grid that contains feasible region .

SAS Code applied:

```
/* Trial run to find optimum solution for flexible storage */

proc nlp all out=nlpresults;

  parms X1 X2 X3 X4
         Y1 Y2 Y3 Y4
         S1 S2 S3 S4;
  bounds 0 <= X1 X2 X3 X4 S1 S2 S3 S4 Y1 Y2 Y3 Y4;
  lincon
    /* System of production equations */
X1=0.75*X4+Y1-100+S1,
X2=2*X3+0.5*X4+Y2-0+S2,
X3=0.25*X4+Y3-200+S3,
X4=Y4-100+S4;

  nlincon nlc1<= 0., nlc2<= 0., nlc3<= 0.,      nlc4<= 0.;

  max profit;

  t=1;Cs=2; r=0.005;
  Sol=100; So2=0; So3=200; So4=100;
  Pin1=80; Pin2=82; Pin3=244; Pin4=162;
  Pout1=82; Pout2=84; Pout3=300; Pout4=190;
  Pout1t=85; Pout2t=87; Pout3t=320; Pout4t=200;
  Cap_s1=2000; Cap_s2=1500;Cap_s3=1000;
  Cap_out1=4000; Cap_out2=2000; Cap_out3=1000; Cap_out4=1000;
  Cap_in1=5000; Cap_in2=3000;
```

```

profit=(Pout1*Y1+Pout2*Y2+Pout3*Y3+Pout4*Y4-Pin1*(X1-S1+So1)-Pin2*(X2-S2+So2)-
Pin3*(X3-S3+So3)-Pin4*(X4-S4+So4))
+(S1*((Pout1t-Cs*t)*exp(-r*t)-Pin1)+S2*((Pout2t-Cs*t)*exp(-r*t)-
Pin2)+S3*((Pout3t-Cs*t)*exp(-r*t)-Pin3)+S4*((Pout4t-Cs*t)*exp(-r*t)-Pin4));

/* Pipe-in exclusive constraints */
nlc1 = X1-Cap_in1;
nlc2 = X2-Cap_in2;

/* Pipe-out non-exclusive constraints */
nlc3=(Y1-S1+So1)+(Y2-S2+So2)+(Y3-S3+So3)+(Y4-S4+So4)-
(Cap_out1+Cap_out2+Cap_out3+Cap_out4);

/*Storage capacity constraints */
/*Choice 0: Trial run assumes storage flexibility */
nlc4=S1+S2+S3+S4-(Cap_s1+Cap_s2+Cap_s3);

run;

```

Exclusive storage corrections and restriction applied to trial run code:

```
/* Choice 1: allocate 3 storage tanks to OO1 (X4) with max S4 gradient in trial run
*/

proc nlp all out=nlpresults_choice1;

    lincon    S1=0,S2=0,S4=0;

    nlc4=S3-(Cap_s1+Cap_s2+Cap_s3);
run;

/* Choice 2: allocate medium tank to OO1 (X3), smallest tank OO2 (X4) with highest
S3, S4 gradients in trial run */
proc nlp all out=nlpresults_choice2;

    lincon S1=0,S2=0;

    nlincon nlc1<= 0., nlc2<= 0., nlc3<= 0.,    nlc4<= 0.,nlc5<= 0.;

    nlc4=S3-Cap_s3;
    nlc5=S4-Cap_s2+Cap_s3;
run;

/* Choice 3: allocate medium tanks to OO1 (X3), smallest tank to OO2 (X4), and
biggest tank to O2 (X2) with highest S3, S4, S2 gradients in trial run */
proc nlp all out=nlpresults_choice3;
    lincon    S1=0;
    nlincon nlc1<= 0., nlc2<= 0., nlc3<= 0.,    nlc4<= 0.,nlc5<= 0.,nlc6<= 0.;
    nlc4=S3-Cap_s3;
    nlc5=S4-Cap_s2;
    nlc6=S2-Cap_s1; run;
```

Results

Table. Some constraints generating the highest profit

Indicator	Trial run	Choice 1	Choice 2	Choice 3
Storage constraints explanation	Flexible storage (not feasible)	3 tanks to 001 (X3) with max S3 gradient in trial run	Medium tank to 001 (X3), smallest tank 002 (X4) with highest S3, S4 gradients in trial run	Medium tank to 001 (X3), smallest tank to 002 (X4), and biggest tank to 02 (X2) with highest S3, S4, S2 gradients in trial run
Storage Constraints formula	$S1+S2+S3+S4 \leq (Cap_s1+Cap_s2+Cap_s3)$;	$S1=0, S2=0, S4=0;$ $S3 \leq (Cap_s1+Cap_s2+Cap_s3)$	$S1=0, S2=0;$ $S3 \leq Cap_s3;$ $S4 \leq Cap_s2;$	$S1=0;$ $S3 \leq Cap_s3;$ $S4 \leq Cap_s2;$ $S2 \leq Cap_s1;$
All constraints check-up	Ok	Ok	Ok	Ok
Solution:				
X1	5000	4600	4800	5000
X2	3000	3000	3000	3000
X3	0	0	0	0
X4	0	0	0	0
Y1	2466	4700	4900	5100
Y2	1434	3000	3000	1000
Y3	0	0	0	0
Y4	0	100	0	0
S1	2634	0	0	0
S2	1566	0	0	2000
S3	200	200	200	200
S4	100	0	100	100
Profit, CAD	\$36,629.77	\$32,682.79	\$33,784.04	\$35,336.16

Final conclusion

Given the input and processing constraints, the business can generate the **maximum profit of \$35,336.16**

Table. An optimum resources' allocation

Oil Product Name	Inventories balance before optimization	Input	Final product out	Inventories balance for the optimization period
Vector notation	S_0	X	Y	S
O1	100	5000	5100	0
O2	0	3000	1000	2000
OO1	200	0	0	200
OO2	100	0	0	100

Future Improvements

1. Functionality for multiple horizons' optimization using forward price curves
2. More flexibility with input parameters (e.g., number of in- and out- pipelines)
3. Automation (by using macro?)
4. A user friendly interface.

Appendix 1. Optimization Criteria

PROC NLP solves

$$\begin{array}{ll} \min_{x \in \mathcal{R}^n} & f(x) \\ \text{subject to} & c_i(x) = 0, \quad i = 1, \dots, m_e \\ & c_i(x) \geq 0, \quad i = m_e + 1, \dots, m \end{array}$$

where f is the objective function and the c_i 's are the constraint functions.

A point x is feasible if it satisfies all the constraints. The feasible region $\mathcal{G}$ is the set of all the feasible points. A feasible point x^* is a global solution of the preceding problem if no point in $\mathcal{G}$ has a smaller function value than $f(x^*)$. A feasible point x^* is a local solution of the problem if there exists some open neighborhood surrounding x^* in that no point has a smaller function value than $f(x^*)$. Nonlinear programming algorithms cannot consistently find global minima. All the algorithms in PROC NLP find a local minimum for this problem. **If you need to check whether the obtained solution is a global minimum, you may have to run PROC NLP with different starting points obtained either at random or by selecting a point on a grid that contains $\mathcal{G}$.**

Every local minimizer x^* of this problem satisfies the following local optimality conditions:

The gradient (vector of first derivatives) $g(x^*) = \nabla f(x^*)$ of the objective function f (projected toward the feasible region if the problem is constrained) at the point x^* is zero.

The Hessian (matrix of second derivatives) $G(x^*) = \nabla^2 f(x^*)$ of the objective function f (projected toward the feasible region $\mathcal{G}$ in the constrained case) at the point x^* is positive definite.

Most of the optimization algorithms in PROC NLP use iterative techniques that result in a sequence of points $x^0, \dots, x^n, \dots$, that converges to a local solution x^* . At the solution, PROC NLP performs tests to confirm that the (projected) gradient is close to zero and that the (projected) Hessian matrix is positive definite.

Karush-Kuhn-Tucker Conditions

An important tool in the analysis and design of algorithms in constrained optimization is the **Lagrangian function**, a linear combination of the objective function and the constraints:

$$L(x, \lambda) = f(x) - \sum_{i=1}^m \lambda_i c_i(x)$$

The coefficients λ_i are called **Lagrange multipliers**. This tool makes it possible to state necessary and sufficient conditions for a local minimum. The various algorithms in PROC NLP create sequences of points, each of which is closer than the previous one to satisfying these conditions.

Assuming that the functions f and c_i are twice continuously differentiable, the point x^* is a **local minimum** of the nonlinear programming problem, if there exists a vector $\lambda^* = (\lambda_1^*, \dots, \lambda_m^*)$ that meets the following conditions.

1. First-order Karush-Kuhn-Tucker conditions:

$$\begin{aligned} c_i(x^*) &= 0, & i &= 1, \dots, m_e \\ c_i(x^*) &\geq 0, & \lambda_i^* &\geq 0, \quad \lambda_i^* c_i(x^*) = 0, & i &= m_e + 1, \dots, m \\ \nabla_x L(x^*, \lambda^*) &= 0 \end{aligned}$$

2. Second-order conditions: Each nonzero vector $y \in \mathcal{H}$ that satisfies

$$y^T \nabla_x c_i(x^*) = 0 \quad \left\{ \begin{array}{l} i = 1, \dots, m_e \\ \forall i \in \{m_e + 1, \dots, m : \lambda_i^* > 0\} \end{array} \right.$$

also satisfies

$$y^T \nabla_x^2 L(x^*, \lambda^*) y > 0$$

Most of the algorithms to solve this problem attempt to find a combination of vectors x and λ for which the gradient of the Lagrangian function with respect to x is zero.

Derivatives

The first- and second-order conditions of optimality are based on first and second derivatives of the objective function f and the constraints .

The gradient vector contains the first derivatives of the objective function f with respect to the parameters $x_1, \dots, x_n$, as follows:

$$g(x) = \nabla f(x) = \left(\frac{\partial f}{\partial x_j} \right)$$

The $n \times n$ symmetric Hessian matrix contains the second derivatives of the objective function f with respect to the parameters $x_1, \dots, x_n$, as follows:

$$G(x) = \nabla^2 f(x) = \left(\frac{\partial^2 f}{\partial x_j \partial x_k} \right)$$

See more details

http://support.sas.com/documentation/cdl/en/ormpug/63352/HTML/default/viewer.htm#ormpug_nlp_sect027.htm

http://support.sas.com/documentation/cdl/en/ormpug/63352/HTML/default/viewer.htm#ormpug_nlp_sect017.htm

Appendix 2. A model for optimum solution for blending – storage business in one time horizon

General model description

With respect to the system arguments X, Y, S :

x.1) Production equation system²:

$$X = AX + Y + (S - S_0)$$

X = a vector of gross oil products (quantity in BBL);

AX = inputs consumed by production (quantity in BBL);

A = matrix of the direct production expenses;

Y = a vector of final oil products to be shipped out (quantity in BBL);

$(S - S_0)$ = change in inventories of oil stored in the storage tanks (quantity in BBL).

Subject to the system constrains

x.2) Input X constrained by pipeline capacity Cap_in for the oil products to be shipped in:

$$X_{in} \leq Cap_in$$

y.1) Output constrained by pipeline capacity Cap_out for the oil products to be shipped out:

$$Y - S + S_0 \leq Cap_out$$

² This model is an extension of input-output model developed by Wassily Leontief who received a Nobel Prize winner in Economics for solving it in 1936.

s.1) Storage capacity Cap_s :

$$S \leq Cap_s$$

Note:

Since storage tanks are mutually exclusive can be devoted only to one type of oil products, there is an extra algorithm for defining the most profitable combination of storage space used:

a) Trial run of the optimization algorithm with flexible constraints to rank the oil products by their gradient (e.g., higher gradient higher profit from the increase of oil product on the storage profit):

$$\text{Maximize } \pi_{Tot} = \pi_B + \pi_S$$

Subject to the full flexibility constraint

$$S_1 + S_2 + \dots + S_k \leq Cap_s$$

where k is a number of the system all input and output oil products;

b) Try different storage constraints that are as close to the trial run's results as possible with general rule of devoting more storage space to the product with higher gradient. Choose the final decision based on the combination generating the highest profit.

$$\text{Maximize } \pi_{Tot} = \pi_B + \pi_S$$

Subject to

$$1^{ST} \text{ run: } S_{1_{\max \text{ gradient}}} \leq Cap_s$$

Nth run: $S1_{\max \text{ gradient}} \leq \text{Cap_s}_{\max}$, $S1 \leq \text{Cap_s}_{\max}$, ..., $SN_{\min \text{ gradient}} \leq \text{Cap_s}_{\min}$

Choose the outcome with highest profit after checking it's compliance with all system restrictions in effect.

See details of this application in the numerical example below.

Non-negativity constraints assuming no waist of the oil products:

x.4) All inputs: $X \geq 0$,

s.2) Stored inventories: $S \geq 0$

y.2) Final products shipped outside of the company's hubs: $Y \geq 0$.

Appendix 3. Wassily Leontieff's Input-Output Model (Nobel prize, 1936):

Basic derivation [\[edit\]](#)

Say that we have an economy with n sectors. Each sector produces x_i units of a single homogeneous good. Assume that the i th sector, in order to produce 1 unit, must use a_{ij} units from sector j . Furthermore, assume that each sector sells some of its output to other sectors (intermediate output) and some of its output to consumers (final output, or final demand). Call final demand in the i th sector d_i . Then we might write

$$x_i = a_{1i}x_1 + a_{2i}x_2 + \dots + a_{ni}x_n + d_i,$$

or total output equals intermediate output plus final output. If we let A be the matrix of coefficients a_{ij} , x be the vector of total output, and d be the vector of final demand, then our expression for the economy becomes

$$x = Ax + d$$

which after re-writing becomes $(I - A)x = d$. If the matrix $I - A$ is invertible then this is a linear system of equations with a unique solution, and so given some final demand vector the required output can be found. Furthermore, if the principal minors of the matrix $I - A$ are all positive (known as the **Hawkins-Simon Condition**), the required output vector x is non-negative.

Example [\[edit\]](#)

Consider an economy with two goods, A and B. The matrix of coefficients and the final demand is given by

$$A = \begin{pmatrix} 0.5 & 0.2 \\ 0.4 & 0.1 \end{pmatrix} \text{ and } d = \begin{pmatrix} 7 \\ 4 \end{pmatrix}.$$

Intuitively, this corresponds to finding the amount of output each sector should produce given that we want 7 units of good A and 4 units of good B. Then solving the system of linear equations derived above gives us

$$x = (I - A)^{-1}d = \begin{pmatrix} 19.19 \\ 12.97 \end{pmatrix}.$$

For practical purposes it might be challenging to actually compute the inverse matrix, given that some input-output tables are in excess of hundreds of sectors.